COESIONE ITALIA 21-27 PIEMONTE | Cofinanziato dall'Unione europea | REGIONE PIEMONTE

Cloud Specialist [D68444-15-2025-23]

Unità Formativa : Programmazione Python

Docente: Massimo PAPA

Titolo argomento: Esercizi sulle classi (OOP) (2)

Intervento realizzato da ITS TECNOLOGIE DELL'INFORMAZIONE E DELLA COMUNICAZIONE

Esercizi sulle classi (OOP)

Indicazioni sulla consegna **3**
Struttura del codice sorgente 4
Esercizi sulle classi (OOP) **6**
Primo Esercizio 6
Secondo Esercizio 6
Terzo Esercizio 6
Esercizi sull'ereditarietà **7**
Quarto Esercizio 7
Quinto Esercizio 7

Indicazioni sulla consegna

Implementare gli algoritmi risolutivi dei seguenti esercizi codificandoli in linguaggio python. Partire analizzando il problema seguendo la seguente traccia:

- Quali sono gli input del problema?
- Quali sono gli output?
- Suddividi il problema in problemi più semplici e se lo ritieni opportuno descrivi la soluzione di ogni sottoproblema con un flow-chart
- Andrai a rappresentare ogni algoritmo che risolve un sottoproblema come una funzione
- La funzione restituisce un valore? Se si di che tipo?
- Ogni funzione accetta una lista di parametri formali? Se si di che tipo?
- Esegui la codifica partendo dal programma principale, al suo interno vai a richiamare le funzioni che poi andrai successivamente a definire.
- Continua la codifica dichiarando e definendo tutte le funzioni prima del programma principale che richiama tutte le altre. In testa alle funzioni scrivi un commento che descrive la funzione stessa , cosa restituisce e quali sono i parametri formali.
- Scrivi la sezione dell'inizializzazione variabili scrivendo in un commento il significa della variabile stessa
- Verifica la codifica utilizzando input di test, cercando di provare anche i casi limite.

Struttura del codice sorgente

Relativamente alle indicazioni di scrittura del codice, utilizza il seguente schema generale:

```
'''
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Testo esercizio

'''

##
## Funzioni:
##


def fun(param1: int, param2: float) -> int:
  '''
     Funzione: fun
      Template per costruire le funzioni
      Parametri formali:
          int Param1 -> descrizione Param1
          float Param2 -> descrizione Param2
      Valore di ritorno:
          int -> descrizione valore di ritorno
  '''
  retValue = 5 # Valore di ritorno della funzione

  return retValue


'''
    Programma principale
    Descrizione sintetica funzionalità
    del programma principale.
'''

# Sezione di input Dati

# Inizializzazioni variabili

# Elaborazione
```

```
# Eventuali sotto processi di Elaborazione

# Sezione di output
```

Esercizi sulle classi (OOP)

Primo Esercizio

Creare una classe Insegnante con attributi nome, età e stipendio, dove stipendio deve essere un attributo privato.

Costruire tutti i metodi getter e setter per gli attributi (anche per quelli pubblici)

Effettuare l'overriding del metodo __str__ in maniera tale che restituisca gli attributi nome e età.

Provare la classe istanziando almeno due oggetti.

Secondo Esercizio

Creare una classe Rettangolo con attributi base e altezza. Costruire tutti i metodi setter e getter per gli attributi. Aggiungere i metodi per il calcolo dell'area e del perimetro.

Implementare un metodo di nome: "contiene" che ha come parametro un oggetto rettangolo. Tale metodo deve restituire true se il rettangolo oggetto chiamante contiene il rettangolo oggetto parametro, false se non lo contiene. Un rettangolo "contiene" un altro quando la sua altezza e la sua base sono maggiori rispettivamente della base e dell'altezza del secondo rettangolo.

Terzo Esercizio

1 - Creare una classe Calcolo con un costruttore di default (senza parametri) che consenta di eseguire vari calcoli su numeri interi.

2 - Creare un metodo chiamato Factorial() che permetta di calcolare il fattoriale di un intero. Testare il metodo istanziando la classe.

3 - Creare un metodo chiamato Sum() che consenta di calcolare la somma dei primi n interi 1 + 2 + 3 + .. + n. Prova questo metodo.

4 - Creare un metodo tableMult() che crea e visualizza la tabellina di un dato intero. Quindi creare un metodo allTablesMult() per visualizzare tutte le tabelline di numeri interi 1, 2, 3, ..., 9.

Esercizi sull'ereditarietà

Quarto Esercizio

Si definisca una classe Persona che abbia i seguenti attributi:

- nome
- indirizzo
- età

Tale classe contiene i seguenti metodi: il costruttore, l'overriding del metodo __str__ e tutti i metodi getter e setter degli attributi.

Si vogliono derivare dalla classe Persona le seguenti classi:

- Studente
- Lavoratore

La prima deve avere gli attributi aggiuntivi:

- Scuola
- Media voti

La seconda deve avere gli attributi aggiuntivi:

- Azienda
- Stipendio

Aggiungere tutti i metodi getter e setter relativi agli attributi aggiuntivi.

Inoltre effettuare l'overriding dei costruttori e del metodo __str__ inserendo gli attributi aggiuntivi.

Provare le tre classi instanziando almeno un oggetto per classe e provando qualche metodo.

Quinto Esercizio

Creare una classe AritmeticaDue con attributi operando1 e operando2. Definire il costruttore utilizzando parametri con valori predefiniti e il metodo __str__.

Aggiungere due metodi uno che restituisca la differenza e l'altro il prodotto dei due operandi. Implementare un terzo metodo che permetta il confronto tra il risultato del prodotto di due oggetti AritmeticaDue (in sostanza indicare se il prodotto è maggiore di quello calcolato nell'oggetto AritmeticaDue passato come parametro).

Derivare dalla classe AritmeticaDue la classe AritmeticaTre aggiungendo l'attributo operando3. Ridefinire il costruttore, il metodo __str__ e i tre metodi differenza, prodotto e confronto. Aggiungere un metodo per il calcolo della somma di tutti gli attributi.

Provare le classi e i metodi implementati.